Temeto Nyilvantartas - Laravel Backend Fejlesztői Dokumentáció

Projektáttekintés

A Temető Nyilvántartás projekt egy temetőkezelő alkalmazás, amely az elhunytakkal, sírhely-nyilvántartással, bérlőkkel és egyéb adminisztratív adatokkal foglalkozik. A backend egy Laravel API-ra épül, amely RESTful endpoint-okat biztosít az adatok kezeléséhez.

Projekt Struktúra

Backend Könyvtár Layout

```
backend/
└── temeto_nyilvantartas_laravel/
    ├── app/
    │   ├── Http/
    │   │   └── Controllers/
    │   │       ├── Settings/
    │   │       │   ├── ProfileController.php
    │   │       │   ├── PasswordController.php
    │   │       │   └── TwoFactorAuthenticationController.php
    │   │       ├── Controller.php (base class)
    │   │       ├── ParcelaController.php
    │   │       ├── SirhelyController.php
    │   │       ├── SorController.php
    │   │       ├── ElhunytController.php
    │   │       ├── BefizetesController.php
    │   │       ├── DokumentumController.php
    │   │       ├── DokumentumTipusController.php
    │   │       ├── SirhelyTipusController.php
    │   │       ├── KozteruletTipusController.php
    │   │       └── ...
```

```
│   └── Models/
│       ├── User.php
│       ├── Parcela.php
│       ├── Sor.php
│       ├── Sirhely.php
│       ├── Sirberlo.php
│       ├── Elhunyt.php
│       ├── Befizetes.php
│       ├── Dokumentum.php
│       ├── Dokumentum_tipus.php
│       ├── Sirhely_tipus.php
│       ├── Kozterulet_tipus.php
│       ├── Telepules.php
│       └── ...
├── routes/
│   ├── api.php (API útvonalak)
│   └── ...
```

Adatbázis Modell

1. Parcela (Parcella)
    - Teljes név: parcella tábla
    - Egy parcella több sorból (Sor) áll

    Parcela (1) —→ (Many) Sor

2. Sor
    - Egy sorhoz több sírhely (Sirhely) tartozik
    - Egy sorhoz egy parcella (Parcela) tartozik

    Sor (Many) —→ (1) Parcela

    Sor (1) —→ (Many) Sirhely

3. Sirhely (Sírhely)
    - Egy sírhelyhez egy vagy több elhunyt (Elhunyt) tartozik
    - Egy sírhelyhez egy sírbérlő (Sirberlo) tartozhat
    - Egy sírhelyhez több dokumentum (Dokumentum) tartozhat
    - Egy sírhelyhez több sírhely típus (Sirhely_tipus) tartozhat

    Sirhely (Many) —→ (1) Sor
    Sirhely (Many) —→ (1) Sirberlo
    Sirhely (1) —→ (Many) Elhunyt
    Sirhely (1) —→ (Many) Dokumentum
    Sirhely (1) —→ (Many) Sirhely_tipus

4. Sirberlo (Sírbérlő)
    - Egy sírbérlőhöz több sírhely (Sirhely) tartozhat
    - Egy sírbérlőhöz több befizetés (Befizetes) tartozhat
    - Egy sírbérlőhöz több dokumentum (Dokumentum) tartozhat
    - Egy sírbérlőhöz egy közterület típus (Kozterulet_tipus) tartozhat
    - Egy sírbérlőhöz egy település (Telepules) tartozhat

    Sirberlo (Many) —→ (1) Kozterulet_tipus
    Sirberlo (Many) —→ (1) Telepules
    Sirberlo (1) —→ (Many) Sirhely
    Sirberlo (1) —→ (Many) Befizetes
    Sirberlo (1) —→ (Many) Dokumentum

5. Elhunyt (Elhunyt)
    - Egy elhunythoz egy sírhely (Sirhely) tartozik
    - Egy elhunythoz egy halotti anyakönyvi kivonat (Dokumentum) tartozhat

    Elhunyt (Many) —→ (1) Sirhely
    Elhunyt (Many) —→ (1) Dokumentum

6. Befizetes (Befizetés)
    - Egy befizetéshez egy sírbérlő (Sirberlo) tartozik
    - Egy befizetéshez egy dokumentum (Dokumentum) tartozhat

    Befizetes (Many) —→ (1) Sirberlo
    Befizetes (Many) —→ (1) Dokumentum

7. Dokumentum (Dokumentum)
    - Egy dokumentumhoz egy vagy több dokumentum típus (Dokumentum_tipus) tartozhat
    - Egy dokumentumhoz egy sírhely (Sirhely) vagy sírbérlő (Sirberlo) tartozhat

    Dokumentum (1) —→ (Many) Dokumentum_tipus

8. Telepules (Település)
    - Egy településhez több sírbérlő (Sirberlo) tartozhat

      Telepules (1) —→ (Many) Sirberlo

9. User (Felhasználó)
    - A Laravel beépített felhasználói modell
    - Kétfaktoros hitelesítésre képes
    - API token-ek (Laravel Sanctum) kezelésére képes

Modellek, és funkcióik

Parcella model

Függvény: sorok()

- Feladata: 1 a többhöz kapcsolatot biztosít a Sor táblával

Sor model

Függvények: parcella(), sirhelyek()

- parcella feladata: több az 1-hez kapcsolatot biztosít a sor és a parcella tábla között
- sirhelyek feladata: 1 a többhöz kapcsolatot biztosít a sor és a sirhelyek tábla között

Sirhely model

- Függvények: sor(), sirberlo(), sirhelyTipusok(), dokumentumok(), elhunytak()
- sor feladata: több az 1-hez kapcsolatot biztosít a sírhely és a sor tábla között
- sirberlo feladata: több az 1-hez kapcsolatot biztosít a sírhely és a sírbérlő tábla között
- sirhelyTipusok feladata: 1 a többhöz kapcsolatot biztosít a sírhely és a sírhelyTípusok tábla között
- dokumentumok feladata: : 1 a többhöz kapcsolatot biztosít a sírhely és a dokumentumok tábla között
- elhunytak feladata: 1 a többhöz kapcsolatot biztosít a sírhely és az elhunytak tábla között

Sirberlo model

Függvények: kozteruletTipus(), telepules(), sirhelyek(), befizetesek(), dokumentumok()

- kozteruletTipus feladata: több az 1-hez kapcsolatot biztosít a Sírbérlő és a közterület típus tábla között
- telepules feladata: több az 1-hez kapcsolatot biztosít a Sírbérlő és a település tábla között
- sirhelyek feladata: 1 a többhöz kapcsolatot biztosít a Sírbérlő és az sírhelyek tábla között
- befizetesek feladata: 1 a többhöz kapcsolatot biztosít a Sírbérlő és az befizetések tábla között
- dokumentumok feladata: 1 a többhöz kapcsolatot biztosít a Sírbérlő és az dokumentumok tábla között

Elhunyt model

Függvények: sirhely(), halottiAyakonyviKivonat()

- sirhely feladata: több az 1-hez kapcsolatot biztosít az elhunyt és a sirhelyek tábla között
- halottiAyakonyviKivonat feladata: több az 1-hez kapcsolatot biztosít az elhunyt és a halotti anyakönyvi kivonat tábla között

Befizetes model

Függvények: sirberlo(), dokumentum()

- sirberlo feladata: több az 1-hez kapcsolatot biztosít az befizetés és a sirberlo tábla között
- dokumentum feladata: több az 1-hez kapcsolatot biztosít az befizetés és a dokumentum tábla között

Dokumentum model

Függvények: dokumentum_tipusok()

- dokumentum_tipusok feladata: 1 a többhöz kapcsolatot biztosít a Dokumentum és a dokumentum típusok tábla között

Dokumentum Tipus model

Függvények: dokumentum(),

- dokumentum feladata: több az 1-hez kapcsolatot biztosít az dokumentum és a dokumentum típusok tábla között

Sirhely_tipus model

Függvények: sirhely()

- sirhely feladata: több az 1-hez kapcsolatot biztosít az sirhely_tipus és a sirhely típusok tábla között

Kozterulet_tipus model

Függvények: sirberlok()

- sirberlok feladata: 1 a többhöz kapcsolatot biztosít a kozterulet_tipus és a sirberlo tábla között

Telepules model

Függvények: sirberlok()

- sirberlok feladata: 1 a többhöz kapcsolatot biztosít a telepules és a sirberlok tábla között

User model

Függvények / tulajdonságok:

use HasFactory, Notifiable, TwoFactorAuthenticatable, HasApiTokens:

- ezek a trait-ek adják a gyári példányosítást, értesítések küldését, kétfaktoros azonosítást és API token kezelést.

$fillable = ['name', 'email', 'password']:

- ezek a mezők tömegesen kitölthetők (mass assignment).

$hidden = ['password', 'two_factor_secret', 'two_factor_recovery_codes', 'remember_token']:

- ezek a mezők nem jelennek meg sorosításkor (pl. JSON-ban).

casts():

- email_verified_at → datetime
- password → hashed (automatikus jelszó-hash)
- two_factor_confirmed_at → datetime

API Útvonalak

1. **Elhunyt (Elhunyt) Végpontok**
   GET /api/elhunytak/nev/{name} → Elhunyt keresése név alapján
   GET /api/elhunytMindenAdata → Az összes elhunyt összes adata
   POST /api/elhunytak → Új elhunyt hozzáadása
   GET /api/elhunytak/count → Elhunytak száma
   GET /api/elhunytak/recent → Legutóbbi 7 elhunyt

2. **Sírhely (Sirhely) Végpontok**
   GET /api/sirhelyek → Összes sírhely listázása (opcionálisan szűrve)
   GET /api/sirhelyek/count → Sírhelyek száma
   GET /api/sirhelyek/{id} → Konkrét sírhely adatai
   POST /api/sirhelyek → Új sírhely létrehozása
   PUT /api/sirhelyek/{id} → Sírhely adatainak frissítése
   DELETE /api/sirhelyek/{id} → Sírhely törlése

3. **Parcella (Parcella) Végpontok**
   GET /api/parcellak → Összes parcella listázása
   GET /api/parcellak/{id} → Konkrét parcella adatai
   POST /api/parcellak → Új parcella létrehozása
   PUT /api/parcellak/{id} → Parcella adatainak frissítése (lehetséges)
   DELETE /api/parcellak/{id} → Parcella törlése (lehetséges)

4. **Sor (Sor) Végpontok**
   GET /api/sorok → Összes sor listázása
   GET /api/sorok/{id} → Konkrét sor adatai
   POST /api/sorok → Új sor létrehozása
   PUT /api/sorok/{id} → Sor adatainak frissítése
   DELETE /api/sorok/{id} → Sor törlése (lehetséges)
   GET /api/parcellak/{id}/sorok → Parcellához tartozó sorok

5. **Befizetés (Befizetes) Végpontok**
   POST /api/befizetesek → Új befizetés rögzítése
   GET /api/befizetesek → Befizetések listázása

6. **Dokumentum Végpontok**
   GET /api/dokumentumok → Összes dokumentum listázása
   POST /api/dokumentumok → Új dokumentum rögzítése

7. **Típus Végpontok**
   GET /api/sirhely_tipusok → Sírhely típusok
   GET /api/dokumentum_tipusok → Dokumentum típusok
   GET /api/kozterulet_tipusok → Közterület típusok
   POST /api/sirhely_tipusok → Sírhely típus hozzáadása
   POST /api/dokumentum_tipusok → Dokumentum típus hozzáadása
   POST /api/kozterulet_tipusok → Közterület típus hozzáadása

8. **Hitelesítés Végpontok**
   GET /api/user → Aktuális felhasználó adatai (auth:sanctum)

Controllerek - Részletes Leírás

<u>ParcelaController</u>

Fő funkciók:

- Parcellák listázása (rendezve a név alapján)
- Parcella keresése ID alapján
- Új parcella létrehozása
- Parcella adatainak frissítése
- Parcella törlésszekciók:

Validációs szabályok:

- nev: Kötelező, szöveg, max 255 karakter

Hibakezelés:

- 404-es válasz, ha a parcella nem található
- 422-es válasz validációs hiba esetén

SirhelyController

Fő funkciók:

- Fő Sírhelyek listázása (opcionális szűrés sor_id alapján)
- Sírhelyek számának lekérdezése
- Konkrét sírhely adatainak lekérdezése
- Új sírhely létrehozása
- Sírhely adatainak frissítése
- Sírhely törlése

Validációs szabályok:

- sor_id: Kötelező, szám, létező sor
- sirkod: Opcionális, szöveg, max 255 karakter
- allapot: Opcionális, szöveg, max 255 karakter
- foto: Opcionális, szöveg, max 255 karakter
- sirberlo_id: Opcionális, szám, létező sírbérlő

Hibakezelés:

- 404-es válasz, ha a sírhely nem található
- 422-es válasz validációs hiba esetén

SorController

Fő funkciók:

- Sorok listázása (opcionális szűrés parcella_id alapján)
- Konkrét sor adatainak lekérdezése
- Új sor létrehozása
- Sor adatainak frissítése
- Sor törlése
- Parcellához tartozó sorok lekérdezése

Validációs szabályok:

- parcella_id: Kötelező, szám, létező parcella
- nev: Opcionális, szöveg, max 255 karakter

Hibakezelés:

- 404-es válasz, ha a sor nem található
- 422-es válasz validációs hiba esetén

ElhunytController

Fő funkciók:

- Elhunytok listázása (név alapján szűrve)
- Összes elhunyt összes adata
- Új elhunyt hozzáadása
- Elhunytak száma
- Legutóbbi 7 elhunyt

Hibakezelés:

- 404-es válasz, ha az elhunyt nem található

BefizetesController

Fő funkciók:

- Befizetések kezelése
- Befizetési adatok rögzítése

DokumentumController

Fő funkciók:

- Dokumentumok listázása
- Új dokumentum rögzítése

Validációs szabályok:

- sirhely_id: Opcionális, szám, létező sírhely
- sirberlo_id: Opcionális, szám, létező sírbérlő
- tipus: Opcionális, szöveg, max 255 karakter
- datum: Opcionális, dátum formátum
- leiras: Opcionális, szöveg, max 1000 karakter

Hibakezelés:

- 422-es válasz validációs hiba esetén

DokumentumTipusController

Fő funkciók:

- Dokumentum típusok listázása
- Új dokumentum típus rögzítése

Validációs szabályok:

- tipus_elnevezese: Kötelező, szöveg, max 255 karakter
- dokumentum_id: Opcionális, szám, létező dokumentum

SirhelyTipusController

Fő funkciók:

- Sírhely típusok listázása
- Új sírhely típus rögzítése

Validációs szabályok:

- sirhely_id: Kötelező, szám, létező sírhely
- nev: Kötelező, szöveg, max 255 karakter

KozteruletTipusController

Fő funkciók:

- Közterület típusok listázása
- Új közterület típus rögzítése

Validációs szabályok:

- nev: Kötelező, szöveg, max 255 karakter

Settings Controllerek

ProfileController

- Profil szerkesztése
- Profil adatainak frissítése
- Fiók törlése

PasswordController

- Jelszó módosítási oldal megjelenítése
- Jelszó frissítése

Validációs szabályok:

- current_password: Kötelező, aktuális jelszó
- password: Kötelező, az alapértelmezett jelszókövetelmények
- password_confirmation: Kötelező, jelszó megerősítése

Hitelesítés és Biztonság

Laravel Sanctum

- A projekt Laravel Sanctum-ot használ az API hitelesítéshez:
- Konfigurációs fájl: config/sanctum.php
- Állapotmentes domének:

Állapotmentes domének:

- localhost, localhost:3000, 127.0.0.1, 127.0.0.1:8000, ::1

Middleware: auth:sanctum

Hitelesített útvonal:

- GET /api/user → Az aktuális felhasználó adatai

Token kezelés:

- Token előtag beállítható a konfigurációban
- Token lejárat beállítható (alapértelmezetten: nincs)
- Személyes hozzáférési token (PAT) támogatás

Jelszókezelés

PasswordController:

- Jelszó megváltoztatása
- Validáció: Aktuális jelszó szükséges
- Megerősítésre van szükség az új jelszó megerősítésére

Jelszó követelmények:

- Minimum hossz: 8 karakter
- Nagybetűt, kisbetűt, számot és szimbólumot kell tartalmaznia

Profilkezelés

ProfileController:

- Profil adatainak szerkesztése
- E-mail cím módosítása
- Fiók törlése

Konfigurációs Beállítások

Wichtig Config Files

config/app.php

- APP_NAME: Alkalmazás neve (env-ből)
- APP_ENV: Alkalmazás környezete (development, production)
- APP_DEBUG: Debug mód (bool)
- APP_URL: Alkalmazás URL-je
- APP_TIMEZONE: Alapértelmezett időzóna (UTC)
- APP_LOCALE: Alkalmazás nyelvezete (en)
- APP_CIPHER: Titkosítási algoritmus (AES-256-CBC)
- APP_KEY: Titkos alkalmazás kulcs (env-ből)

config/auth.php

- GUARD: Hitelesítési guard (web)
- PROVIDER: Felhasználói provider (users)
- DRIVER: Adatbázis driver (eloquent)
- MODEL: Felhasználó modell (App\Models\User)
- PASSWORD_RESET_TOKEN_TABLE: Jelszóvisszaállítási token tábla neve
- PASSWORD_RESET_TIMEOUT: Jelszóvisszaállítási token lejárati ideje (perc)

config/sanctum.php

- STATEFUL_DOMAINS: Állapotmentes domének listája
- GUARD: Autentikációs guard ([web])
- EXPIRATION: Token lejárati ideje (null = nincs lejárat)
- TOKEN_PREFIX: Token előtag az SCPv.nél
- MIDDLEWARE:
- Sanctum middleware-k

config/cache.php

- CACHE_STORE: Alapértelmezett cache driver (database)
- CACHE_TABLE: Cache tábla neve
- CACHE_KEY_PREFIX: Cache kulcs előtag

config/queue.php

- QUEUE_CONNECTION: Alapértelmezett queue driver (database)
- DB_QUEUE_CONNECTION: Adatbázis connection (null)
- DB_QUEUE_TABLE: Queue tábla neve (jobs)
- DB_QUEUE_RETRY_AFTER: Újrapróbálkozás után (másodperc)

config/m MAIL_MAILER: Alapértelmezett mailer (log)

- MAIL_HOST: SMTP host
- MAIL_PORT: SMTP port
- MAIL_USERNAME: SMTP felhasználónév
- MAIL_PASSWORD: SMTP jelszó
- MAIL_FROM_ADDRESS: Alapértelmezett feladó e-mail
- MAIL_FROM_NAME: Alapértelmezett feladó névail.php

config/services.php

- POSTMARK_API_KEY: Postmark API kulcs
- RESEND_API_KEY: Resend API kulcs
- AWS_ACCESS_KEY_ID: AWS hozzáférési kulcs
- AWS_SECRET_ACCESS_KEY: AWS titok
- SLACK_BOT_USER_OAUTH_TOKEN: Slack bot token

Hibakezelés és Validáció

Validációs Hibaüzenetek

Valamennyi controller Validator::make() függvényt használ az alábbi formátumban:

Dokumentációs leírás (validáció):

- A kérés adataiból ($request->all()) validátort hozunk létre, ahol
- a mezők szabályait a második tömb tartalmazza (pl. field_name => validation_rules),
- a mezőkhöz tartozó magyar hibaüzenetek a harmadik tömbben adhatók meg (pl. field_name.rule => 'Hungarian error message').
- Ha a validáció megbukik ($validator->fails()), a válasz JSON:
- success: false
- message: "Az adatok nem megfelelőek!"
- errors: a validátor hibáinak tömbje ($validator->errors()->toArray()),
- HTTP státuszkód: 422.

Validációs Szabályok:

- required: Kötelező mező
- string: Szöveg típus
- integer: Egész szám
- max:255: Maximum 255 karakter
- date: Dátum formátum
- exists:table,column: Létezik az adatbázisban
- nullable: Opcionális mező

HTTP Státuszkódok:

- 200: Sikeres GET/PUT
- 201: Sikeres POST (létrehozás)
- 202: Sikeres POST/PUT (módosítás)
- 404: Nem található
- 422: Validációs hiba
- 401: Hitelesítésre szorul
- 403: Hozzáférés megtagadva
- 500: Szerver hiba

Hibakezelési Mintázatok:

- Nem található (404): Ha a $record nem létezik, JSON választ ad vissza message mezővel: „Nincs rekord ezzel az id-val.” és HTTP 404 kóddal.
- Validációs hiba (422): Ha a $validator->fails(), JSON választ küld success: false, message: Az adatok nem megfelelőek!, valamint az összes hiba errors mezőben, HTTP 422 kóddal.
- Sikeres válasz (201): Siker esetén JSON választ ad success: true, message: Sikeres művelet., és a visszaadott adatot data mezőben, HTTP 201 kóddal.

Éles Környezetre Telepítés

- Composer telepítése
- composer install --no-dev

- Environment fájl másolása
- cp .env.example .env

- Adatbázis migrálása
- php artisan migrate –force

.env

- DB_CONNECTION=mysql
- DB_HOST=127.0.0.1
- DB_PORT=3306
- DB_DATABASE=temeto_nyilvantartas
- DB_USERNAME=root
- DB_PASSWORD=

Laravel és PHP Verzió Információ

Laravel Verzió

Aktuális verzió: Laravel 12

Laravel 12 Jellemzői:

Laravel 12 a Laravel keretrendszer legújabb fő kiadása, amely több javítást és új funkciót tartalmaz:

Fő Jellemzők:

- Modern PHP 8.2+ szintaksis és funkciók
- Inertia.js integráció az SPA-hoz

- Filepond file upload komponens
- Tanstack Query integrációja
- Improved error handling
- Optimális teljesítmény
- Laravel Herd támogatás
- Pest testing keretrendszer integráció
- Laravel Pulse (alkalmazás monitoring)
- Database encryption

Alapértelmezett Csomagok Laravel 12-ben:

- laravel/framework: ^12.0
- laravel/sanctum: ^4.0
- laravel/fortify: ^1.18
- laravel/tinker: ^2.8
- laravel/breeze: ^2.0

PHP Verzió

Aktuális verzió: PHP 8.2 - 8.3+

PHP 8.2 Jellemzői

PHP 8.2 jelentős teljesítményi és biztonsági fejlesztéseket hozta:

- Readonly osztályok, immutabilitás könnyen.
- DNF típusok összetett uniók.
- Dinamikus property tiltás alapból.
- true/false önálló típus.
- Random\Randomizer erős RNG.
- Trait konstansok támogatva.
- mysqli execute_query egyszerűbb bind.
- Opcache/JIT finomítás gyorsabb.